

Pamantasan ng Lungsod ng Maynila

College of Information Systems and Technology Management

MASTER JAVA SWING

DEVELOPMENT GUIDE

FOR THE NSTP INFORMATION SYSTEM

Foundations, Phase 1 & Phase 2 Edition

Project	Development of an Integrated Information System for NSTP, PLM
Platform	Java SE 21 Java Swing MySQL via JDBC
Database	nstp_infosystem (MySQL / XAMPP)
Group	GROUP FIFTH UNIT
Members	Famero Capulong Tinoko Vallite De Leon Baterbonia
Instructor	Prof. Darwin C. Co
Date	May 2026 CONFIDENTIAL

This document is the official master development guide for Group Fifth Unit's capstone project.

TABLE OF CONTENTS

SECTION 1: INTRODUCTION TO JAVA SWING	5
1.1 What Is Java Swing?	5
1.2 Why Java Swing for Desktop Information Systems?	5
1.3 Advantages for School Information Systems	5
1.4 Key Java Swing Components	6
JFrame — The Application Window	6
JPanel — Container for Components	6
JButton — Clickable Action Triggers	6
JLabel — Text and Image Display	7
JTable — Data Display Grid	7
JTextField — Single-Line Text Input	7
JComboBox — Dropdown Selection	8
JPasswordField — Secure Password Input	8
JMenuBar — Application Menu System	8
JOptionPane — Dialog Boxes	8
JScrollPane — Scrollable Content	9
1.5 Event-Driven Programming Concept	9
1.6 MVC Concept in Swing Projects	9
1.7 How Swing Maps to Our NSTP Modules	10
1.8 Setup Best Practices and Beginner Mistakes to Avoid	10
SECTION 2: GUIDES AND TIPS FOR JAVA SWING PROJECT	12
2.1 Project Coding Standards	12
2.2 Naming Conventions	12
2.3 Folder Organization	13
2.4 UI Consistency Rules	13
2.5 Validation Practices	13
2.6 Exception Handling Policy	14
2.7 JDBC Connection Safety	14
2.8 Common Imports Reference	14
2.9 Performance Tips	15
2.10 Team Collaboration Workflow	15
2.11 Version Control Workflow (Recommended)	15
SECTION 3: FEATURES OF THE NSTP INFORMATION SYSTEM	16
3.1 Authentication Features	16
3.1.1 Login	16
3.1.2 Logout and Session Clearing	16

3.1.3 Role-Based Access Control (RBAC).....	16
3.1.4 Password Change	16
3.2 Administrator Features	16
3.2.1 Student Management	17
3.2.2 User Account Management.....	17
3.2.3 Announcements Management	17
3.2.4 Calendar Event Management.....	17
3.2.5 Reports and Data Export	17
3.3 Attendance Features	17
3.4 Engagement Features (CWTS).....	18
3.5 Performance Features (ROTC).....	18
3.6 Points Features	18
3.7 Demerit Features (ROTC).....	18
3.8 Reporting Features.....	19
3.9 Security Features	19
SECTION 4: NECESSARY SOFTWARE REQUIREMENTS.....	20
4.1 JDK 21 Setup Verification	20
4.2 Adding MySQL Connector/J to the Project	21
In IntelliJ IDEA:	21
In Apache NetBeans:.....	21
SECTION 5: PHASE 1 — FOUNDATION SETUP, PROJECT STRUCTURE, DATABASE CONNECTIVITY	22
5.1 Task 5.1 — Create Project Package Structure.....	22
5.2 Task 5.2 — Create Java Project Folders	23
5.3 Task 5.3 — Required Libraries	23
5.4 Task 5.4 — DBConnection.java (Full Production Code)	23
5.5 Task 5.5 — SessionManager.java (Full Production Code).....	25
5.6 Task 5.6 — Model Classes (Full Code Examples)	27
User.java.....	27
Student.java	28
AttendanceRecord.java	29
5.7 Task 5.7 — Main.java Application Entry Point.....	30
5.8 Task 5.8 — LoginFrame.java Starter Skeleton.....	31
SECTION 6: PHASE 2 — DECLARATIONS, VARIABLES, INITIALIZATION, GLOBAL STRUCTURE	35
6.1 Login Module Variables.....	35
6.2 Database Module Variables	35
6.3 Comprehensive Variable Reference by Module	36

Admin Module	36
Attendance Module	36
Performance Module (ROTC)	37
Engagement Module (CWTS)	37
Points Module	38
Reports Module	39
System-Wide Variables	39
SECTION 7: IMPORTANT ENGINEERING STANDARDS	41
7.1 PreparedStatement-Only Policy	41
7.2 Layer Separation Rules	41
7.3 Testing Checklist (Before Marking a Module Complete)	41
7.4 Backup Protocol	42
SECTION 8: TEAM EXECUTION PLAN	43
8.1 Suggested Member Roles	43
8.2 Daily Team Workflow	43
8.3 Database Ownership Rules	43
SECTION 9: FINAL NOTE AND CONTINUATION PLAN	45
What Will Be Covered in Later Guides	45

SECTION 1: INTRODUCTION TO JAVA SWING

This section provides a comprehensive introduction to Java Swing, the GUI toolkit that powers the NSTP Information System desktop application. Every team member, regardless of role, must read and understand this section before writing any user interface code.

1.1 What Is Java Swing?

Java Swing is a platform-independent Graphical User Interface (GUI) toolkit included in the Java Standard Edition (SE) library. It provides a rich set of components — windows, buttons, tables, menus, text fields — that allow developers to build fully functional desktop applications without any additional installation.

Swing is part of the `javax.swing` package hierarchy and is built on top of the older Abstract Window Toolkit (AWT). Unlike AWT, which relies on native operating system components, Swing components are rendered entirely by Java, making them look and behave consistently across Windows, macOS, and Linux.

Key distinction: Swing is a **pure Java** solution — no external frameworks, no web technologies, no dependencies beyond the JDK itself.

1.2 Why Java Swing for Desktop Information Systems?

For a school-based information system that must run reliably on institutional computers without an internet connection, Java Swing is one of the most appropriate technology choices available. The following reasons justify this decision for the NSTP project:

- **Offline capability:** The system connects to a local MySQL database via JDBC. No server, no browser, no network dependency.
- **Institutional compatibility:** Most Philippine university computer laboratories run Windows with JDK pre-installed or easily installed.
- **Simplicity of deployment:** A runnable JAR file is sufficient to distribute the entire application.
- **Performance:** Swing applications start quickly and consume minimal system resources compared to web-based alternatives.
- **Academic suitability:** Swing is the standard desktop GUI technology taught in Java software engineering and system development courses.
- **Mature ecosystem:** Swing has been in production since 1997, with extensive documentation, tutorials, and community support.
- **Full control:** Developers control every pixel of the interface without relying on third-party layout engines or CSS frameworks.

1.3 Advantages for School Information Systems

Why Schools Choose Desktop Applications

Many Philippine universities prefer desktop over web-based systems for administrative tools because: (1) they do not require continuous internet access, (2) they do not expose sensitive student data over the public internet, (3) they are simpler to maintain — no web server configuration, no hosting costs, and (4) they perform better on older hardware common in school laboratories.

For the NSTP context specifically, the advantages are:

- Student records never leave the local campus network during normal operation.
- The system runs during NSTP sessions without dependence on WiFi or LAN stability.
- The administrator can manage the database directly through phpMyAdmin, which is familiar to ISITM students.
- The project aligns perfectly with the course curriculum: Java OOP, SQL databases, JDBC integration, and software engineering principles.

1.4 Key Java Swing Components

The following components form the building blocks of every panel and module in the NSTP Information System. Study each one carefully — they will appear in your code every day.

JFrame — The Application Window

JFrame is the top-level container that represents a window in a Swing application. Every major window in the system — LoginFrame, MainFrame, StudentFrame — is a JFrame subclass.

JFrame — Basic Usage

```
public class LoginFrame extends JFrame {  
    public LoginFrame() {  
        setTitle("NSTP Information System");  
        setSize(500, 400);  
        setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);  
        setLocationRelativeTo(null); // center on screen  
        setVisible(true);  
    }  
}
```

- Always call `setDefaultCloseOperation()` to prevent the window from hiding without closing resources.
- `setLocationRelativeTo(null)` centers the window on the screen.
- `setVisible(true)` must be called after all components are added, inside `SwingUtilities.invokeLater()`.

JPanel — Container for Components

JPanel is the most widely used intermediate container in Swing. Every section of a window — the sidebar, the header bar, the form area — is a JPanel. Panels are nested inside each other to build complex layouts.

JPanel with GridBagLayout

```
JPanel formPanel = new JPanel(new GridBagLayout());  
formPanel.setBackground(new Color(0x1B3A6B)); // navy background  
GridBagConstraints gbc = new GridBagConstraints();  
gbc.insets = new Insets(8, 8, 8, 8);  
gbc.fill = GridBagConstraints.HORIZONTAL;  
gbc.gridx = 0; gbc.gridy = 0;  
formPanel.add(new JLabel("Username:"), gbc);
```

JButton — Clickable Action Triggers

JButton represents a clickable push button. It fires ActionEvents when clicked, which are handled by ActionListeners attached to it. Every form in the NSTP system has Save, Cancel, and action-specific buttons.

JButton with ActionListener

```
JButton btnLogin = new JButton("Login");
btnLogin.setBackground(new Color(0x2E5FA3));
btnLogin.setForeground(Color.WHITE);
btnLogin.setFont(new Font("Arial", Font.BOLD, 13));
btnLogin.addActionListener(e -> handleLogin()); // Lambda listener
```

JLabel — Text and Image Display

JLabel displays non-editable text or images. Used for field labels, titles, status indicators, and decorative elements throughout the system.

JLabel Examples

```
JLabel lblTitle = new JLabel("NSTP Information System");
lblTitle.setFont(new Font("Arial", Font.BOLD, 18));
lblTitle.setForeground(new Color(0xB8860B)); // gold color

// Label with image icon
JLabel lblIcon = new JLabel(new ImageIcon("assets/logo.png"));
```

JTable — Data Display Grid

JTable displays tabular data in rows and columns. It is the primary data display component for student lists, attendance records, transaction histories, and all data grids in the system.

JTable with DefaultTableModel

```
String[] columns = {"Student ID", "Full Name", "Program", "Status"};
DefaultTableModel model = new DefaultTableModel(columns, 0) {
    @Override public boolean isCellEditable(int row, int col) {
        return false; // make table read-only
    }
};
JTable table = new JTable(model);
table.setRowHeight(28);
table.getTableHeader().setBackground(new Color(0x1B3A6B));
table.getTableHeader().setForeground(Color.WHITE);
JScrollPane scrollPane = new JScrollPane(table);
```

JTextField — Single-Line Text Input

JTextField accepts user keyboard input for single-line text. Used for username, search bars, activity names, and all short-form input fields.

JTextField Setup

```
JTextField txtUsername = new JTextField(20); // 20 columns wide
txtUsername.setFont(new Font("Arial", Font.PLAIN, 12));
txtUsername.setBorder(BorderFactory.createLineBorder(Color.GRAY));

// Live search using DocumentListener
txtSearch.getDocument().addDocumentListener(new DocumentListener() {
    public void insertUpdate(DocumentEvent e) { filterTable(); }
});
```

```
public void removeUpdate(DocumentEvent e) { filterTable(); }
public void changedUpdate(DocumentEvent e) { filterTable(); }
});
```

JComboBox — Dropdown Selection

JComboBox provides a dropdown list for selecting one value from a predefined set. Used for role selection, program type (CWTS/ROTC), status selection, and severity levels.

JComboBox — Program Selector

```
String[] programs = {"CWTS", "ROTC"};
JComboBox<String> cboProgram = new JComboBox<>(programs);
cboProgram.setSelectedIndex(0);

// Populate from database
JComboBox<String> cboStudents = new JComboBox<>();
for (Student s : studentDAO.getAllStudents()) {
    cboStudents.addItem(s.getStudentId() + " - " + s.getFullName());
}
```

JPasswordField — Secure Password Input

JPasswordField masks the characters typed by the user, making it suitable for password entry. Always use this instead of JTextField for any password input in the system.

JPasswordField — Secure Retrieval

```
JPasswordField txtPassword = new JPasswordField(20);
txtPassword.setEchoChar('*');

// CORRECT: use getPassword() not getText()
char[] passChars = txtPassword.getPassword();
String password = new String(passChars);
java.util.Arrays.fill(passChars, '0'); // clear sensitive data
```

JMenuBar — Application Menu System

JMenuBar provides a horizontal menu bar at the top of a JFrame. JMenu represents each dropdown menu, and JMenuItem represents individual clickable options. Useful for File, Tools, Help menus.

JOptionPane — Dialog Boxes

JOptionPane creates standard dialog windows for messages, confirmations, input prompts, and error alerts. It is heavily used throughout the system for user feedback.

JOptionPane — Common Patterns

```
// Information dialog
JOptionPane.showMessageDialog(this, "Record saved.", "Success",
    JOptionPane.INFORMATION_MESSAGE);

// Error dialog
JOptionPane.showMessageDialog(this, "Invalid credentials.", "Login Failed",
    JOptionPane.ERROR_MESSAGE);

// Confirmation dialog
int result = JOptionPane.showConfirmDialog(this,
```



```
"Are you sure you want to delete this record?",
"Confirm Delete", JOptionPane.YES_NO_OPTION);
if (result == JOptionPane.YES_OPTION) { deleteRecord(); }
```

JScrollPane — Scrollable Content

JScrollPane wraps any component that may exceed its visible area, providing scrollbars automatically. Always wrap JTable and long JPanel contents in a JScrollPane.

JScrollPane — Wrapping a Table

```
JTable tblStudents = new JTable(model);
JScrollPane scrollPane = new JScrollPane(tblStudents);
scrollPane.setVerticalScrollBarPolicy(
    JScrollPane.VERTICAL_SCROLLBAR_ALWAYS);
scrollPane.setPreferredSize(new Dimension(700, 350));
```

1.5 Event-Driven Programming Concept

Java Swing applications are event-driven. This means the program waits for user actions — mouse clicks, key presses, window resizing — and responds to each one through a listener callback mechanism.

The key components of event handling are:

- **Event Source:** The component that generates the event (e.g., JButton).
- **Event Object:** An object representing the event that occurred (e.g., ActionEvent, MouseEvent).
- **Event Listener:** An interface that defines what should happen when the event fires (e.g., ActionListener).
- **Handler Method:** The method (like actionPerformed()) that is called when the event fires.

Event Thread Safety

All Swing GUI updates must occur on the Event Dispatch Thread (EDT). Always wrap GUI creation in `SwingUtilities.invokeLater()` -> { ... }. Never update a JTable or JLabel from a background thread directly — use `SwingUtilities.invokeLater()` or a `SwingWorker` to dispatch updates back to the EDT.

1.6 MVC Concept in Swing Projects

The Model-View-Controller (MVC) architecture separates an application into three interconnected layers. While Swing does not enforce MVC strictly, the NSTP project must follow a strict layered separation inspired by MVC:

Layer	Responsibility	NSTP Package
Model	Holds data structure and state. Plain Java objects representing database rows.	model (User.java, Student.java)
View	The user interface panels, forms, tables, and dialogs the user interacts with.	ui.login, ui.admin, ui.student

Layer	Responsibility	NSTP Package
Controller	Handles business logic, validation, and orchestrates between View and DAO.	service (ReportService, NotificationService)
Data Access	SQL queries, JDBC operations. Not strictly MVC but essential layer.	dao (UserDAO, AttendanceDAO)

Golden Rule

Never write SQL queries directly inside a JPanel or JButton ActionListener. All database access must go through a DAO class. All business logic must go through a Service class. UI classes must only call DAO or Service methods and display the results.

1.7 How Swing Maps to Our NSTP Modules

System Feature	Primary Swing Components Used
Login System	JFrame, JTextField, JPasswordField, JButton, JOptionPane
Admin Dashboard	JFrame, JPanel, JLabel (summary cards), BorderLayout
Student List	JTable, DefaultTableModel, JScrollPane, JTextField (search)
Attendance Logging	JTable, JComboBox, JSpinner, JButton (save), JDialog
Engagement Records	JTable, JTextField, JComboBox, JProgressBar
Performance Entries	JTable, JComboBox, JTextArea (remarks), JLabel
Points Leaderboard	JTable, JLabel (rank badge), JScrollPane
Announcements	JPanel (card), BoxLayout, JScrollPane, JLabel (priority)
Reports & Export	JComboBox (type), JTextArea (preview), JFileChooser
Notification Center	JPanel (notification card), JScrollPane, javax.swing.Timer

1.8 Setup Best Practices and Beginner Mistakes to Avoid

MISTAKE #1 — Calling setVisible() Before Adding Components

Always add all components to a JFrame BEFORE calling setVisible(true). If you call setVisible() too early, components may not appear or the layout may look broken.

MISTAKE #2 — Building GUI Outside invokeLater()

All Swing GUI creation must happen inside SwingUtilities.invokeLater(). Writing new JFrame() directly in the main() method without invokeLater() is unsafe and can cause threading issues.

MISTAKE #3 — Using getText() on JPasswordField

JPasswordField.getText() is deprecated and insecure. Always use getPassword() which returns a char[] array. After using the password, fill the array with zeros to clear it from memory.

MISTAKE #4 — Writing SQL Inside JButton ActionListeners

SQL queries written directly in button handlers cannot be tested, reused, or debugged efficiently. All SQL must be in DAO classes. This is enforced team policy for the NSTP project.

MISTAKE #5 — Not Wrapping JTable in JScrollPane

A JTable added directly to a JPanel without a JScrollPane will not show column headers and will clip data. Always: JScrollPane scroll = new JScrollPane(table); panel.add(scroll);

SECTION 2: GUIDES AND TIPS FOR JAVA SWING PROJECT

This section establishes the official coding standards, conventions, and workflow rules for all team members. Consistency in naming, structure, and practices ensures that every team member can read and extend each other's code without confusion.

2.1 Project Coding Standards

- Use Java SE 21 features but avoid preview or experimental features.
- All class files must begin with the package declaration, followed by imports, then the class body.
- No class should exceed 500 lines. If a class grows beyond this, refactor into helper classes.
- Every method must do exactly one thing. Long methods must be broken into smaller private methods.
- All database operations are wrapped in try-catch blocks catching SQLException.
- All user-facing error messages use JOptionPane.showMessageDialog() — never System.err.println() alone.
- No magic numbers. All constants (port, database name, colors, thresholds) go in a Constants or UIStyle class.

2.2 Naming Conventions

Element	Convention	Example
Class names	PascalCase	AttendanceAdminPanel, UserDAO
Method names	camelCase verb	getStudentById(), saveAttendance()
Variable names	camelCase noun	studentId, currentRole
Constants	UPPER_SNAKE_CASE	DB_URL, MAX_ATTEMPTS
JTextField vars	txt prefix	txtUsername, txtSearch
JButton vars	btn prefix	btnSave, btnDelete
JTable vars	tbl prefix	tblStudents, tblAttendance
JComboBox vars	cbo prefix	cboProgram, cboStatus
JLabel vars	lbl prefix	lblStatus, lblTitle
JPanel vars	pnl prefix	pnlForm, pnlHeader
JDialog vars	dlg prefix	dlgAddUser, dlgConfirm
DAO classes	Entity + DAO suffix	StudentDAO, AttendanceDAO
Model classes	Entity name only	Student, AttendanceRecord
Service classes	Feature + Service	ReportService, NotificationService

2.3 Folder Organization

Package / Folder	Contents
ui.login	LoginFrame.java — the login window and its sub-components
ui.admin	MainFrame, all admin panels (UserManagementPanel, AttendanceAdminPanel, etc.)
ui.student	StudentFrame, all student panels (StudentDashboardPanel, AttendanceStudentPanel, etc.)
ui.shared	Panels used by multiple roles: CalendarPanel, AnnouncementStudentPanel, LeaderboardPanel
model	Plain Java data objects: User, Student, AttendanceRecord, EngagementActivity, etc.
dao	Database access: UserDAO, StudentDAO, AttendanceDAO, EngagementDAO, PerformanceDAO, etc.
service	Business logic: ReportService, NotificationService, AttendanceRateService
session	SessionManager.java — the Singleton session state holder
util	DBConnection, UIStyle, DateHelper, FileExporter, PasswordUtil, Constants
resources/	Application icon, logo image, font files
sql/	nstp_infosystem.sql schema backup, nstp_demo_database.sql
reports/	Exported .txt report files generated at runtime

2.4 UI Consistency Rules

- All panels use the color constants defined in UIStyle.java — never hardcode hex colors inline.
- Form labels are always right-aligned. Input fields are always the same width within a form.
- All action buttons (Save, Delete, Edit) have a minimum size of 120x35 pixels.
- JTables always have a row height of 28 pixels and a navy-colored header.
- All error dialogs use JOptionPane.ERROR_MESSAGE. Success dialogs use JOptionPane.INFORMATION_MESSAGE.
- Section titles use Arial 16pt Bold in navy color. Form labels use Arial 12pt. Table data uses Arial 12pt.

2.5 Validation Practices

- Validate all form fields before calling any DAO method. Never pass empty strings or nulls to the database.
- Trim whitespace from all text field inputs: txtName.getText().trim()
- Check that numeric fields contain valid numbers using Integer.parseInt() inside a try-catch.
- For date pickers, validate that the date is not in the future for past-event entries (attendance, drills).

- Check for duplicate records before INSERT — use a DAO method that queries for existing records with the same key fields.
- Validate password fields: minimum 8 characters, at least one letter and one number.

2.6 Exception Handling Policy

Standard Exception Handling Pattern for DAOs

```
public List<Student> getAllStudents() {
    List<Student> list = new ArrayList<>();
    String sql = "SELECT * FROM students ORDER BY full_name";
    try (Connection conn = DBConnection.getConnection();
        PreparedStatement pst = conn.prepareStatement(sql);
        ResultSet rs = pst.executeQuery()) {
        while (rs.next()) {
            list.add(mapResultSet(rs));
        }
    } catch (SQLException e) {
        e.printStackTrace(); // log for developer
        JOptionPane.showMessageDialog(null,
            "Database error: " + e.getMessage(),
            "Error", JOptionPane.ERROR_MESSAGE);
    }
    return list;
}
```

2.7 JDBC Connection Safety

- Always use try-with-resources to automatically close Connection, PreparedStatement, and ResultSet.
- Never keep a Connection object open for longer than a single operation. Open, use, close.
- Always use PreparedStatement — never String concatenation to build SQL queries.
- Check that getConnection() does not return null before using the connection.
- Database credentials (URL, username, password) must only appear in DBConnection.java — never in any other class.

2.8 Common Imports Reference

Import	What It Provides	Used In
java.awt.*	Color, Font, Dimension, GridBagLayout, BorderLayout, FlowLayout, GridLayout, Insets	All UI panels
java.awt.event.*	ActionListener, MouseListener, DocumentListener, ActionEvent, MouseEvent	All panels with buttons or live search
javax.swing.*	JFrame, JPanel, JButton, JLabel, JTable, JTextField, JComboBox, JScrollPane, JOptionPane, JDialog, BoxLayout, CardLayout, SwingUtilities, SwingWorker, Timer	All UI classes

Import	What It Provides	Used In
javax.swing.table.*	DefaultTableModel, TableRowSorter, DefaultTableCellRenderer	All panels with JTable
java.sql.*	Connection, PreparedStatement, ResultSet, SQLException, DriverManager	All DAO classes and DBConnection
java.time.*	LocalDate, LocalDateTime, DateTimeFormatter	Attendance, Calendar, Notifications
java.util.*	ArrayList, List, HashMap, Map, Arrays	All model and DAO classes
java.io.*	FileWriter, BufferedWriter, File, IOException	ReportService, FileExporter

2.9 Performance Tips

- Use SQL aggregation (SUM, COUNT, AVG) in queries — never fetch all rows and compute in Java loops.
- Use PreparedStatement batch execution for bulk inserts (attendance sessions with many students).
- Use SwingWorker for any operation that takes more than 100ms — loading reports, bulk queries.
- Load JTable data asynchronously and show a loading indicator while the query runs.
- Cache frequently read, rarely changed data (program list, role list) in memory — no need to re-query every time.

2.10 Team Collaboration Workflow

1. Before starting work each day, sync with the shared SQL backup file and confirm your local database is up to date.
2. Each team member works on their assigned module. Do not edit files assigned to another member without coordination.
3. Changes to DBConnection.java and SessionManager.java require team approval — these affect all modules.
4. Test your module against live database data before marking a task as complete.
5. At the end of each development day, export the database as a .sql backup and save it to the shared folder.

2.11 Version Control Workflow (Recommended)

- Use Git with a shared repository (GitHub or GitLab) if the team is comfortable with it.
- Each team member works on a separate branch named after their module: feature/attendance, feature/points, etc.
- Merge to the main branch only after testing your module. One team member reviews all merges.
- Commit messages must describe what was done: 'Add AttendanceDAO.saveSession() with batch insert'.
- If not using Git, maintain a shared folder with timestamped zip backups of the src/ folder each day.

SECTION 3: FEATURES OF THE NSTP INFORMATION SYSTEM

This section catalogues all system features, organized by category. Each feature entry explains its purpose, which user roles interact with it, which database tables it reads or writes, why the feature is necessary for the NSTP context, and how it could be expanded in future development iterations.

3.1 Authentication Features

3.1.1 Login

Attribute	Detail
Purpose	Authenticates users against the users table and establishes a session.
Roles	All roles: Admin, CWTS Student, ROTC Student, ROTC Senior Officer
Database Tables	users (read), students (join for name display)
Why Necessary	Prevents unauthorized access to student records and administrative functions.
Future Expansion	Two-factor authentication via email OTP, biometric login for school terminals.

3.1.2 Logout and Session Clearing

Clicking Logout calls `SessionManager.clearSession()`, disposes the current frame, and reopens the `LoginFrame`. All session data is erased from memory. No student data remains visible after logout. This is critical when shared computers are used in laboratory environments.

3.1.3 Role-Based Access Control (RBAC)

Role	Accessible Modules
Admin	All modules: Users, Students, Attendance, Engagement, Performance, Points, Demerits, Announcements, Calendar, Reports, Feedback, Profile
CWTS Student	Dashboard, Attendance (view), Engagement (view), Points (view), Notifications, Calendar, Announcements, Profile, Feedback
ROTC Student	Dashboard, Attendance (view), Performance (view), Points (view), Demerits (view), Notifications, Calendar, Announcements, Profile
ROTC Senior Officer	Dashboard, Performance (entry), Demerit (assignment), Calendar, Announcements, Logout

3.1.4 Password Change

Available to all authenticated users via the Profile panel. The current password is verified against the SHA-256 hash stored in the database before the new password is accepted. The new password must meet the minimum security requirements: 8 characters, at least one letter and one number.

3.2 Administrator Features

3.2.1 Student Management

The administrator can add new student records, edit existing records (program, section, year level, contact, email), and deactivate student accounts. This module writes to the students and users tables.

3.2.2 User Account Management

The administrator creates system user accounts, links each account to a student profile, assigns a role, sets an initial password, and can deactivate accounts when a student completes the program or leaves.

Database tables affected: users (INSERT, UPDATE), students (JOIN for display).

3.2.3 Announcements Management

Administrators post announcements with a title, body, audience filter (All / CWTS / ROTC), priority level (Normal / Important / Urgent), and optional pin flag. Students see only announcements targeted to their program. Pinned and Urgent announcements are visually elevated.

Database tables: announcements (INSERT, SELECT).

3.2.4 Calendar Event Management

Administrators create calendar events assigned to a program type. Events appear on the student-facing monthly calendar as colored dots. The notification system monitors upcoming events and generates alerts for students 3 days in advance.

Database tables: calendar_events (INSERT, SELECT).

3.2.5 Reports and Data Export

The reports module generates formatted text reports for individual students or entire classes. Report types include: Attendance Summary, Engagement Hours Summary, Performance Trend, Points Standing, Demerit Summary, and At-Risk Student List. Reports are previewed in the application and exported to .txt files.

3.3 Attendance Features

Feature	Description	DB Table
Session Logging	Admin logs a session date and name, assigns status to each enrolled student.	attendance
Bulk Actions	Mark All Present / Mark All Absent buttons apply a status to all rows at once.	attendance
Duplicate Guard	System checks for existing records with the same session name and date before saving.	attendance
Student View	Color-coded table showing all attendance records with rate and progress bar.	attendance
Dispute Submission	Students submit corrections for Absent/Late entries. Saved to disputes table.	disputes
Attendance Rate	Computed as (Present count / Total count) x 100 per student.	attendance

Feature	Description	DB Table
At-Risk Flag	Attendance below 75% triggers a visual warning on the student dashboard.	attendance

3.4 Engagement Features (CWTS)

- Activity Logging: Admin records community service activities — name, category, date, location, hours.
- Bulk Assignment: Admin assigns one activity to multiple students simultaneously using a multi-select list.
- Verification Flag: Admin marks records as verified. Only verified hours count toward the completion threshold.
- Student Progress View: Shows total verified hours and a JProgressBar toward the required minimum (default: 20 hours).
- Completion Badge: Displays when a CWTS student has met or exceeded the required engagement hours.
- Database tables: engagement (INSERT, SELECT, UPDATE).

3.5 Performance Features (ROTC)

- Drill Rating Entry: ROTC Senior Officer or Admin assigns drill ratings: Excellent, Proficient, Satisfactory, Needs Improvement, Unsatisfactory.
- Average Score: Computed from all ratings using numeric equivalents (Excellent=5, Proficient=4, etc.).
- At-Risk Detection: If the last 3 consecutive ratings are Needs Improvement or Unsatisfactory, an at-risk flag is activated.
- Color-Coded History: Student view uses green-to-red color gradient based on rating level.
- Database tables: performance (INSERT, SELECT).

3.6 Points Features

- Award and Deduction: Admin awards or deducts points with a reason and date. Both write to the points table.
- Net Balance: Computed as SUM(award amounts) - SUM(deduction amounts) per student.
- Negative Balance Guard: System blocks a deduction if it would result in a negative balance.
- Leaderboard: Top 10 students per program ranked by net points balance. Updated on every load.
- Transaction History: Full list of all transactions with color coding (green=award, red=deduction).
- Database tables: points (INSERT, SELECT).

3.7 Demerit Features (ROTC)

- Demerit Assignment: ROTC Senior or Admin assigns demerits with reason, severity (Minor/Moderate/Major), notes, and officer name.
- Severity Weighting: Minor=1 point, Moderate=2 points, Major=3 points. Weighted total computed in SQL.

- Status Tracking: Demerits can be Active, Appealed, or Dismissed.
- Standing Badge: Student view shows 'Safe Standing' or 'Demerit Warning' based on weighted total threshold.
- Database tables: demerits (INSERT, UPDATE, SELECT).

3.8 Reporting Features

Report Type		Data Source	Output
Individual Student Report		All tables for one student_id	Full summary of attendance, points, performance/engagement
Class Attendance Report		attendance JOIN students	All students with attendance rates for a session or period
Engagement (CWTS)	Summary	engagement program=CWTS WHERE	Hours per student, completion status
Performance (ROTC)	Summary	performance JOIN students	Average rating, at-risk flags per student
Points Standing		points GROUP BY student_id	Net balance ranked for CWTS and ROTC separately
At-Risk Student List		attendance, engagement performance,	Students below thresholds across all monitored metrics

3.9 Security Features

- Input Validation: All form fields validated before DAO calls. Empty, null, and out-of-range values are rejected.
- SQL Injection Prevention: 100% PreparedStatement usage — no string concatenation in SQL.
- Password Hashing: SHA-256 one-way hash. Plain text passwords are never stored.
- Duplicate Record Prevention: DAO checks for existing records before INSERT for attendance sessions.
- Role-Based Panel Access: Panel constructors verify the current role from SessionManager.
- Session Invalidation: clearSession() on logout removes all user identity data from memory.
- Database Error Isolation: Raw SQL error messages are never shown to users — always replaced with friendly messages.

SECTION 4: NECESSARY SOFTWARE REQUIREMENTS

Every team member must have all of the following software installed and working before writing any project code. Installation order matters — follow the sequence below.

#	Software	Version	Purpose	Download
1	Java Development Kit (JDK)	21 LTS	Compiles and runs the Java application. The entire project is built on Java SE 21.	oracle.com or adoptium.net
2	Apache NetBeans IDE OR IntelliJ IDEA Community	Latest Stable	Primary development environment. IntelliJ recommended for its superior code intelligence and refactoring tools.	netbeans.apache.org or jetbrains.com
3	XAMPP	8.2+ (PHP 8.2)	Provides local Apache web server and MySQL Server for the database. phpMyAdmin is included.	apachefriends.org
4	MySQL Connector/J (JDBC Driver)	8.0 or 8.2	JAR file that allows Java to communicate with MySQL via JDBC. Must be added to project libraries.	dev.mysql.com/downloads
5	Git (Optional but Recommended)	Latest	Version control. Allows team members to collaborate on code and track changes safely.	git-scm.com
6	phpMyAdmin (included in XAMPP)	Built-in	Web-based MySQL management tool. Used to create tables, run SQL, and manage the nstp_infosystem database.	Included with XAMPP

Installation Order

Install in this exact order: (1) JDK 21, (2) IDE (NetBeans or IntelliJ), (3) XAMPP, (4) Add MySQL Connector JAR to your IDE project. Do not install XAMPP before the JDK, as some environments may conflict.

4.1 JDK 21 Setup Verification

Verify JDK Installation in Terminal / Command Prompt

```
# Windows (Command Prompt)
java -version
# Expected output: openjdk version "21.x.x" ...

javac -version
# Expected output: javac 21.x.x

# If not recognized, add JDK to PATH:
# Windows: System Properties > Environment Variables > PATH > Add C:\Program Files\Java\jdk-21\bin
```

```
# macOS/Linux: export JAVA_HOME=/usr/lib/jvm/java-21-openjdk; export  
PATH=$JAVA_HOME/bin:$PATH
```

4.2 Adding MySQL Connector/J to the Project

In IntelliJ IDEA:

6. File > Project Structure (Ctrl+Alt+Shift+S)
7. Go to Modules > Dependencies tab
8. Click the '+' button > JARs or Directories
9. Navigate to the mysql-connector-j-8.x.x.jar file and select it
10. Click Apply and OK

In Apache NetBeans:

11. Right-click the project in the Projects panel > Properties
12. Go to Libraries > Add JAR/Folder
13. Select the mysql-connector-j-8.x.x.jar file
14. Click OK

Test the Connection

After adding the JAR, create a temporary test class and write: `import java.sql.Connection;` If the IDE does not show a red import error, the connector is correctly linked. Proceed to the `DBConnection.java` setup in Section 5.

SECTION 5: PHASE 1 — FOUNDATION SETUP, PROJECT STRUCTURE, DATABASE CONNECTIVITY

Phase 1 Goal

By the end of Phase 1, your team will have: a working Java project with the correct package structure, a live JDBC connection to the nstp_infosystem MySQL database, all model classes created, the SessionManager Singleton implemented, and a runnable application entry point that opens the LoginFrame skeleton.

5.1 Task 5.1 — Create Project Package Structure

Organize the Java project exactly as described below. This structure enforces separation of concerns and makes it easy for each team member to find and work on their assigned files.

Package	Responsibility	Key Classes
ui.login	Login and authentication windows and dialogs	LoginFrame
ui.admin	Main admin window and all admin-facing panels	MainFrame, UserManagementPanel, AttendanceAdminPanel, EngagementAdminPanel, PerformanceAdminPanel, PointsAdminPanel, DemeritAdminPanel, AnnouncementAdminPanel, ReportsPanel, FeedbackAdminPanel
ui.student	Student main window and all student-facing panels	StudentFrame, StudentDashboardPanel, AttendanceStudentPanel, EngagementStudentPanel, PerformanceStudentPanel, PointsStudentPanel, NotificationPanel, ProfilePanel, FeedbackStudentPanel
ui.shared	Panels shared by multiple roles	CalendarPanel, AnnouncementStudentPanel, LeaderboardPanel
model	Plain Java data objects (no database logic)	User, Student, AttendanceRecord, EngagementActivity, PerformanceEntry, PointTransaction, Demerit
dao	SQL queries and JDBC operations per module	UserDAO, StudentDAO, AttendanceDAO, EngagementDAO, PerformanceDAO, PointsDAO, DemeritDAO, AnnouncementDAO, NotificationDAO, CalendarDAO, FeedbackDAO
service	Business logic: rate computation, flagging, report generation	ReportService, NotificationService, AttendanceService, PerformanceService, EngagementService

Package	Responsibility	Key Classes
session	Session state Singleton	SessionManager
util	Reusable helper classes and constants	DBConnection, UIStyle, DateHelper, FileExporter, PasswordUtil, Constants

5.2 Task 5.2 — Create Java Project Folders

Recommended Project Folder Structure	
NSTP_InfoSystem/	
├─ src/	
│ └─ main/	
│ └─ java/	
│ ├─ ui/login/	(LoginFrame.java)
│ ├─ ui/admin/	(MainFrame.java, all admin panels)
│ ├─ ui/student/	(StudentFrame.java, all student panels)
│ ├─ ui/shared/	(CalendarPanel.java, LeaderboardPanel.java)
│ ├─ model/	(User.java, Student.java, etc.)
│ ├─ dao/	(UserDAO.java, AttendanceDAO.java, etc.)
│ ├─ service/	(ReportService.java, etc.)
│ ├─ session/	(SessionManager.java)
│ └─ util/	(DBConnection.java, UIStyle.java, etc.)
├─ resources/	
│ ├─ logo.png	
│ └─ app_icon.png	
├─ sql/	
│ ├─ nstp_infosystem.sql	(schema backup)
│ └─ nstp_demo_database.sql	(demo data backup)
└─ reports/	(runtime-generated .txt exports)

5.3 Task 5.3 — Required Libraries

The only external library required for the NSTP project is the MySQL Connector/J JAR file. Follow the instructions in Section 4.2 to add it to your IDE project. Confirm the import works before proceeding.

5.4 Task 5.4 — DBConnection.java (Full Production Code)

DBConnection is the central JDBC utility class. Every DAO class calls DBConnection.getConnection() to obtain a database connection. This class encapsulates all connection details so they never appear anywhere else in the codebase.

util/DBConnection.java — Complete Implementation
<pre>package util; import java.sql.Connection; import java.sql.DriverManager; import java.sql.SQLException; /**</pre>

```

* DBConnection — Central JDBC connection utility for NSTP Information System.
* All DAO classes use this class to obtain and release database connections.
* Database: nstp_infosystem (MySQL via XAMPP, localhost:3306)
*/
public class DBConnection {

    // — Connection Parameters —————
    private static final String DB_URL =
        "jdbc:mysql://localhost:3306/nstp_infosystem" +
        "?useSSL=false&allowPublicKeyRetrieval=true&serverTimezone=Asia/Manila";
    private static final String DB_USER = "root";
    private static final String DB_PASSWORD = ""; // Default XAMPP: blank

    /** Private constructor — prevents instantiation. Use static methods only. */
    private DBConnection() {}

    /**
     * Opens and returns a new JDBC Connection to nstp_infosystem.
     * The caller is responsible for closing this connection.
     * Best practice: use inside a try-with-resources block.
     *
     * @return Connection object, or null if connection fails
     */
    public static Connection getConnection() {
        try {
            // Load the MySQL JDBC driver (required for some environments)
            Class.forName("com.mysql.cj.jdbc.Driver");
            return DriverManager.getConnection(DB_URL, DB_USER, DB_PASSWORD);
        } catch (ClassNotFoundException e) {
            System.err.println("[DBConnection] MySQL driver not found: " +
e.getMessage());
            return null;
        } catch (SQLException e) {
            System.err.println("[DBConnection] Connection failed: " + e.getMessage());
            return null;
        }
    }

    /**
     * Safely closes a Connection if it is open.
     * Use this in finally blocks when NOT using try-with-resources.
     *
     * @param conn The Connection to close (may be null — handled safely)
     */
    public static void closeConnection(Connection conn) {
        if (conn != null) {
            try {
                if (!conn.isClosed()) {
                    conn.close();
                }
            } catch (SQLException e) {
                System.err.println("[DBConnection] Close failed: " + e.getMessage());
            }
        }
    }
}

```



```

    }

    /**
     * Tests the database connection and prints the result to the console.
     * Call this once during development to verify JDBC setup is correct.
     */
    public static void testConnection() {
        try (Connection conn = getConnection()) {
            if (conn != null && !conn.isClosed()) {
                System.out.println("[DBConnection] Database connection successful.");
            } else {
                System.out.println("[DBConnection] Connection returned null.");
            }
        } catch (SQLException e) {
            System.err.println("[DBConnection] Test failed: " + e.getMessage());
        }
    }
}

```

Line-by-Line Explanation

DB_URL: The JDBC connection string. 'jdbc:mysql:/' is the protocol. 'localhost:3306' is the MySQL server address and port. 'nstp_infosystem' is the database name. The parameters disable SSL (not needed locally), enable public key retrieval (required for MySQL 8.x), and set the timezone to Manila. DB_USER and DB_PASSWORD: Default XAMPP root credentials. If your team set a password, update DB_PASSWORD here only. Class.forName(): Explicitly loads the MySQL driver class. Required in some environments. getConnection(): Opens the connection. Returns null on failure to allow callers to handle gracefully. closeConnection(): Safely closes a connection, checking for null and already-closed state.

5.5 Task 5.5 — SessionManager.java (Full Production Code)

SessionManager is a Singleton class that holds the identity of the currently logged-in user. It is the single source of truth for authentication state. All panels read from SessionManager to determine what data to display and what actions to permit.

session/SessionManager.java — Complete Implementation

```

package session;

/**
 * SessionManager — Singleton session state holder for NSTP Information System.
 * Stores the identity and role of the currently authenticated user.
 * All panels read from this class to determine access and display data.
 */
public class SessionManager {

    // — Singleton Instance —————
    private static SessionManager instance;

    // — Session Fields —————

```

```
private int    currentUserId;
private String currentUsername;
private String currentRole;      // Admin / CWTS_Student / ROTC_Student / ROTC_Senior
private int    currentStudentId; // 0 if Admin (no linked student profile)
private String currentFullName;
private String currentProgram;   // CWTS or ROTC (null for Admin)
private boolean isLoggedInIn;

/** Private constructor – enforces Singleton pattern. */
private SessionManager() {
    clearSession();
}

/**
 * Returns the single SessionManager instance.
 * Creates it on first call (lazy initialization).
 */
public static synchronized SessionManager getInstance() {
    if (instance == null) {
        instance = new SessionManager();
    }
    return instance;
}

// — Session Control —————

/**
 * Sets all session fields upon successful login.
 * Call this immediately after validating credentials in UserDao.
 */
public void setSession(int userId, String username, String role,
                      int studentId, String fullName, String program) {
    this.currentUserId    = userId;
    this.currentUsername  = username;
    this.currentRole      = role;
    this.currentStudentId = studentId;
    this.currentFullName  = fullName;
    this.currentProgram   = program;
    this.isLoggedInIn     = true;
}

/**
 * Clears all session data. Call this on logout.
 * Resets all fields to default safe values.
 */
public void clearSession() {
    this.currentUserId    = 0;
    this.currentUsername  = null;
    this.currentRole      = null;
    this.currentStudentId = 0;
    this.currentFullName  = null;
    this.currentProgram   = null;
    this.isLoggedInIn     = false;
}
```

```
// — Convenience Role Checks —————
public boolean isAdmin()      { return "Admin".equals(currentRole); }
public boolean isCWTSStudent() { return "CWTS_Student".equals(currentRole); }
public boolean isROTCStudent() { return "ROTC_Student".equals(currentRole); }
public boolean isROTCSenior() { return "ROTC_Senior".equals(currentRole); }

// — Getters —————
public int      getCurrentUserId()    { return currentUserId; }
public String   getCurrentUsername()  { return currentUsername; }
public String   getCurrentRole()      { return currentRole; }
public int      getCurrentStudentId() { return currentStudentId; }
public String   getCurrentFullName()  { return currentFullName; }
public String   getCurrentProgram()   { return currentProgram; }
public boolean  isLoggedIn()          { return isLoggedIn; }
}
```

How to Use SessionManager in Any Panel

To get the current user's role from any panel: `String role = SessionManager.getInstance().getCurrentRole();` To get the student ID for database queries: `int sid = SessionManager.getInstance().getCurrentStudentId();` To check if an admin is logged in: `if (SessionManager.getInstance().isAdmin()) { showAdminMenu(); }`

5.6 Task 5.6 — Model Classes (Full Code Examples)

Model classes are plain Java objects (POJOs) that represent a single database row. They have no SQL logic, no Swing components, and no business rules — just fields, a constructor, and getters/setters.

User.java

model/User.java — Complete Implementation

```
package model;

/**
 * User — Represents one record from the 'users' database table.
 * Fields map directly to the users table columns.
 */
public class User {
    private int      userId;
    private String   username;
    private String   passwordHash;
    private String   role;           // Admin / CWTS_Student / ROTC_Student / ROTC_Senior
    private int      studentId;      // FK to students table (0 for Admin)
    private int      isActive;       // 1 = active, 0 = deactivated

    public User() {}

    public User(int userId, String username, String passwordHash,
                String role, int studentId, int isActive) {
        this.userId      = userId;
        this.username    = username;
    }
}
```

```

        this.passwordHash = passwordHash;
        this.role          = role;
        this.studentId     = studentId;
        this.isActive      = isActive;
    }

    // Getters
    public int      getUserId()      { return userId; }
    public String   getUsername()    { return username; }
    public String   getPasswordHash() { return passwordHash; }
    public String   getRole()        { return role; }
    public int      getStudentId()    { return studentId; }
    public int      getIsActive()     { return isActive; }

    // Setters
    public void setUserId(int userId)      { this.userId = userId; }
    public void setUsername(String username) { this.username = username; }
    public void setPasswordHash(String hash) { this.passwordHash = hash; }
    public void setRole(String role)       { this.role = role; }
    public void setStudentId(int studentId) { this.studentId = studentId; }
    public void setIsActive(int isActive)  { this.isActive = isActive; }

    @Override
    public String toString() {
        return "User{id=" + userId + ", user='" + username + "', role='" + role + "'}";
    }
}

```

Student.java

model/Student.java — Complete Implementation

```

package model;

/**
 * Student — Represents one record from the 'students' database table.
 * The central entity — all attendance, engagement, performance,
 * points, and demerit records reference student_id as a foreign key.
 */
public class Student {
    private int      studentId;
    private String   fullName;
    private String   program;      // CWTS or ROTC
    private String   yearLevel;
    private String   section;
    private String   contactNo;
    private String   email;

    public Student() {}

    public Student(int studentId, String fullName, String program,
                   String yearLevel, String section, String contactNo, String email) {
        this.studentId = studentId;
        this.fullName   = fullName;
    }
}

```

```

        this.program    = program;
        this.yearLevel = yearLevel;
        this.section    = section;
        this.contactNo  = contactNo;
        this.email      = email;
    }

    // Getters
    public int    getStudentId() { return studentId; }
    public String getFullName()  { return fullName; }
    public String getProgram()   { return program; }
    public String getYearLevel() { return yearLevel; }
    public String getSection()   { return section; }
    public String getContactNo() { return contactNo; }
    public String getEmail()     { return email; }

    // Setters
    public void setStudentId(int studentId) { this.studentId = studentId; }
    public void setFullName(String fullName) { this.fullName = fullName; }
    public void setProgram(String program)   { this.program = program; }
    public void setYearLevel(String yearLevel) { this.yearLevel = yearLevel; }
    public void setSection(String section)   { this.section = section; }
    public void setContactNo(String contactNo) { this.contactNo = contactNo; }
    public void setEmail(String email)       { this.email = email; }

    @Override
    public String toString() {
        return studentId + " - " + fullName + " (" + program + ")";
    }
}

```

AttendanceRecord.java

model/AttendanceRecord.java — Complete Implementation

```

package model;

import java.time.LocalDate;

/**
 * AttendanceRecord — Represents one record from the 'attendance' table.
 * One record = one student's status for one session.
 */
public class AttendanceRecord {
    private int    recordId;
    private int    studentId;
    private String sessionName;
    private LocalDate sessionDate;
    private String status;    // Present / Absent / Late / Excused
    private String remarks;

    public AttendanceRecord() {}

    public AttendanceRecord(int recordId, int studentId, String sessionName,

```

```

        LocalDate sessionDate, String status, String remarks) {
    this.recordId    = recordId;
    this.studentId   = studentId;
    this.sessionName = sessionName;
    this.sessionDate = sessionDate;
    this.status      = status;
    this.remarks     = remarks;
}

// Getters
public int      getRecordId()    { return recordId; }
public int      getStudentId()   { return studentId; }
public String   getSessionName() { return sessionName; }
public LocalDate getSessionDate() { return sessionDate; }
public String   getStatus()      { return status; }
public String   getRemarks()    { return remarks; }

// Setters
public void setRecordId(int recordId)    { this.recordId = recordId; }
public void setStudentId(int studentId)  { this.studentId = studentId; }
public void setSessionName(String sessionName){ this.sessionName = sessionName; }
public void setSessionDate(LocalDate date) { this.sessionDate = date; }
public void setStatus(String status)      { this.status = status; }
public void setRemarks(String remarks)   { this.remarks = remarks; }
}

```

5.7 Task 5.7 — Main.java Application Entry Point

Main.java — Application Entry Point (Complete)

```

import session.SessionManager;
import ui.login.LoginFrame;

import javax.swing.*;

/**
 * Main — Entry point for the NSTP Information System.
 * Launches the LoginFrame on the Event Dispatch Thread (EDT).
 * Never build Swing components outside SwingUtilities.invokeLater().
 */
public class Main {

    public static void main(String[] args) {
        // Set system look and feel for native-style rendering
        try {
            UIManager.setLookAndFeel(UIManager.getSystemLookAndFeelClassName());
        } catch (Exception e) {
            System.err.println("Look and feel not set: " + e.getMessage());
        }

        // All GUI must be created on the Event Dispatch Thread
        SwingUtilities.invokeLater(() -> {
            // Initialize SessionManager (clears any stale session state)
            SessionManager.getInstance().clearSession();
        });
    }
}

```

```

        // Launch the login screen
        LoginFrame loginFrame = new LoginFrame();
        loginFrame.setVisible(true);
    });
}
}

```

Why invokeLater()? [View](#)

Swing is single-threaded. All GUI creation, modification, and event handling must happen on the Event Dispatch Thread (EDT). `SwingUtilities.invokeLater()` schedules the GUI code to run on the EDT safely. Without it, you may get random layout issues, flickering, or crashes in multi-core environments.

5.8 Task 5.8 — LoginFrame.java Starter Skeleton

ui/login/LoginFrame.java — Starter Skeleton

```

package ui.login;

import dao.UserDAO;
import model.User;
import session.SessionManager;
import ui.admin.MainFrame;
import ui.student.StudentFrame;

import javax.swing.*;
import java.awt.*;

/**
 * LoginFrame — The main authentication window for NSTP Information System.
 * Validates credentials, routes to the correct dashboard by role.
 */
public class LoginFrame extends JFrame {

    private JTextField txtUsername;
    private JPasswordField txtPassword;
    private JButton btnLogin;
    private JButton btnClear;
    private int failedAttempts = 0;
    private static final int MAX_ATTEMPTS = 3;

    public LoginFrame() {
        initComponents();
        setupListeners();
    }

    private void initComponents() {
        setTitle("NSTP Information System — Pamantasan ng Lungsod ng Maynila");
        setSize(480, 380);
        setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        setLocationRelativeTo(null);
    }
}

```

```

        setResizable(false);

        // Main panel with navy background
        JPanel mainPanel = new JPanel(new GridBagLayout());
        mainPanel.setBackground(new Color(0x1B3A6B));
        mainPanel.setBorder(BorderFactory.createEmptyBorder(30, 50, 30, 50));

        GridBagConstraints gbc = new GridBagConstraints();
        gbc.insets = new Insets(8, 8, 8, 8);
        gbc.fill = GridBagConstraints.HORIZONTAL;

        // Title label
        JLabel lblTitle = new JLabel("NSTP Information System", SwingConstants.CENTER);
        lblTitle.setFont(new Font("Arial", Font.BOLD, 18));
        lblTitle.setForeground(new Color(0xB8860B));
        gbc.gridx = 0; gbc.gridy = 0; gbc.gridwidth = 2;
        mainPanel.add(lblTitle, gbc);

        // Subtitle
        JLabel lblSub = new JLabel("Pamantasan ng Lungsod ng Maynila",
        SwingConstants.CENTER);
        lblSub.setFont(new Font("Arial", Font.PLAIN, 11));
        lblSub.setForeground(Color.LIGHT_GRAY);
        gbc.gridy = 1;
        mainPanel.add(lblSub, gbc);

        // Username field
        gbc.gridwidth = 1; gbc.gridy = 2; gbc.gridx = 0;
        JLabel lblUser = new JLabel("Username:");
        lblUser.setForeground(Color.WHITE);
        lblUser.setFont(new Font("Arial", Font.BOLD, 12));
        mainPanel.add(lblUser, gbc);

        gbc.gridx = 1;
        txtUsername = new JTextField(18);
        mainPanel.add(txtUsername, gbc);

        // Password field
        gbc.gridy = 3; gbc.gridx = 0;
        JLabel lblPass = new JLabel("Password:");
        lblPass.setForeground(Color.WHITE);
        lblPass.setFont(new Font("Arial", Font.BOLD, 12));
        mainPanel.add(lblPass, gbc);

        gbc.gridx = 1;
        txtPassword = new JPasswordField(18);
        mainPanel.add(txtPassword, gbc);

        // Button panel
        JPanel btnPanel = new JPanel(new FlowLayout(FlowLayout.CENTER, 15, 0));
        btnPanel.setOpaque(false);

        btnLogin = new JButton("Login");
    
```



```

        btnLogin.setBackground(new Color(0x2E5FA3));
        btnLogin.setForeground(Color.WHITE);
        btnLogin.setFont(new Font("Arial", Font.BOLD, 13));
        btnLogin.setPreferredSize(new Dimension(120, 35));
        btnLogin.setFocusPainted(false);

        btnClear = new JButton("Clear");
        btnClear.setFont(new Font("Arial", Font.PLAIN, 12));
        btnClear.setPreferredSize(new Dimension(100, 35));

        btnPanel.add(btnLogin);
        btnPanel.add(btnClear);

        gbc.gridy = 4; gbc.gridx = 0; gbc.gridwidth = 2;
        mainPanel.add(btnPanel, gbc);

        add(mainPanel);
    }

    private void setupListeners() {
        btnLogin.addActionListener(e -> handleLogin());
        btnClear.addActionListener(e -> {
            txtUsername.setText("");
            txtPassword.setText("");
            txtUsername.requestFocus();
        });
        // Allow Enter key to submit login
        txtPassword.addActionListener(e -> handleLogin());
    }

    private void handleLogin() {
        if (failedAttempts >= MAX_ATTEMPTS) {
            JOptionPane.showMessageDialog(this,
                "Too many failed attempts. Please restart the application.",
                "Access Locked", JOptionPane.ERROR_MESSAGE);
            return;
        }

        String username = txtUsername.getText().trim();
        char[] passChars = txtPassword.getPassword();
        String password = new String(passChars);
        java.util.Arrays.fill(passChars, '0'); // clear sensitive data

        if (username.isEmpty() || password.isEmpty()) {
            JOptionPane.showMessageDialog(this,
                "Please enter both username and password.",
                "Input Required", JOptionPane.WARNING_MESSAGE);
            return;
        }

        // TODO Phase 2: Replace with real UserDAO.validateLogin() call
        // UserDAO dao = new UserDAO();
        // User user = dao.validateLogin(username, password);
    }

```

```
        // Placeholder: simulate failed login for now
        JOptionPane.showMessageDialog(this,
            "Login will be implemented in Phase 2.",
            "Placeholder", JOptionPane.INFORMATION_MESSAGE);
    }

    private void openDashboard(User user) {
        switch (user.getRole()) {
            case "Admin":
            case "ROTC_Senior":
                new MainFrame().setVisible(true);
                break;
            default:
                new StudentFrame().setVisible(true);
                break;
        }
        dispose(); // close login window
    }
}
```

SECTION 6: PHASE 2 — DECLARATIONS, VARIABLES, INITIALIZATION, GLOBAL STRUCTURE

This section focuses exclusively on variable declarations: what variables are needed, their types, their scope, and how they map to the database. Understanding variable structure before coding each module ensures every team member uses consistent naming and prevents naming conflicts.

6.1 Login Module Variables

Variable	Type	Scope	DB Field	Purpose
txtUsername	JTextField	Panel instance	users.username	Captures username input for authentication
txtPassword	JPasswordField	Panel instance	users.password_hash	Captures password input securely
btnLogin	JButton	Panel instance	N/A	Triggers the authentication process
btnClear	JButton	Panel instance	N/A	Clears username and password fields
failedAttempts	int	Panel instance, starts at 0	N/A	Tracks consecutive failed logins this session
MAX_ATTEMPTS	static final int = 3	Class constant	N/A	Maximum failed attempts before lockout
currentUser	User (model)	Local in handler	users table row	Holds the authenticated User object after login

6.2 Database Module Variables

Standard DAO Variable Declarations (used in every DAO method)

```
// These are LOCAL variables – declared inside each method, not as class fields
Connection      conn = null; // JDBC connection (opened per operation)
PreparedStatement pst = null; // Parameterized SQL statement
ResultSet       rs   = null; // Rows returned by a SELECT query

// Correct pattern using try-with-resources:
try (Connection conn = DBConnection.getConnection();
     PreparedStatement pst = conn.prepareStatement(sql);
```

```

        ResultSet rs = pst.executeQuery() {
        // process rs
    } catch (SQLException e) {
        e.printStackTrace();
    }

```

6.3 Comprehensive Variable Reference by Module

Admin Module

Variable	Type	DB Mapping	Purpose
tblStudents	JTable	students (SELECT *)	Displays all student records in the admin panel
modelStudents	DefaultTableModel	N/A	Data source for tblStudents
tblUsers	JTable	users JOIN students	Displays all user accounts
cboProgram	JComboBox<String>	students.program ENUM	Filters or sets CWTS / ROTC selection
cboRole	JComboBox<String>	users.role ENUM	Selects role when creating/editing user accounts
btnAdd	JButton	Triggers INSERT	Opens the Add Student or Add User dialog
btnEdit	JButton	Triggers UPDATE	Opens the Edit dialog for the selected row
btnDeactivate	JButton	users.is_active = 0	Deactivates the selected user account
txtSearch	JTextField	Filters query or model	Live-search field above student or user table
selectedStudentId	int	students.student_id PK	Holds the ID of the row selected in tblStudents

Attendance Module

Variable	Type	DB Mapping	Purpose
txtSessionName	JTextField	attendance.session_name	Admin enters the name of the attendance session
spnSessionDate	JSpinner (SpinnerDateModel)	attendance.session_date	Admin selects the date of the session
tblAttendance	JTable	attendance JOIN students	Shows all students with a status dropdown per row
cboStatus	JComboBox<String>[]	attendance.status ENUM	One per row: Present / Absent / Late / Excused

Variable	Type	DB Mapping	Purpose
btnMarkAllPresent	JButton	Bulk UPDATE	Sets all rows to Present status instantly
btnSaveSession	JButton	Triggers batch INSERT	Saves all attendance records for the session
currentAttendanceRate	double	Computed attendance from	Attendance rate percentage for progress bar display
pbarAttendance	JProgressBar	Computed field	Visual indicator of student's attendance rate
selectedRecordId	int	attendance.record_id PK	ID of the selected attendance row for dispute submission

Performance Module (ROTC)

Variable	Type	DB Mapping	Purpose
txtDrillName	JTextField	performance.drill_name	Name of the drill or exercise being rated
spnDrillDate	JSpinner	performance.drill_date	Date the drill was conducted
cboRating	JComboBox<String>	performance.rating ENUM	Drill performance rating selection
txtaRemarks	JTextArea	performance.remarks	Officer's narrative remarks about the drill
txtOfficerName	JTextField	performance.officer_name	Name of the submitting officer
averageRating	double	Computed performance from	Numeric average of all drill ratings
isAtRisk	boolean	Computed (last 3 ratings)	True if last 3 ratings are low — triggers warning badge
tblPerformance	JTable	performance students JOIN	Color-coded history table for student or admin view

Engagement Module (CWTS)

Variable	Type	DB Mapping	Purpose
txtActivityName	JTextField	engagement.activity_name	Name of the community service activity

Variable	Type	DB Mapping	Purpose
cboCategory	JComboBox<String>	engagement.category	Type of activity: Community Service, Environmental, etc.
spnActivityDate	JSpinner	engagement.activity_date	Date the activity was conducted
txtLocation	JTextField	engagement.location	Location where the activity took place
spnHours	JSpinner (SpinnerNumberModel)	engagement.hours_rendered	Number of hours rendered (0.5 to 24.0)
totalHoursRendered	double	SUM(hours_rendered) WHERE verified=1	Total verified hours for progress calculation
REQUIRED_HOURS	static final double = 20.0	Business rule constant	Minimum hours required for completion badge
pbarEngagement	JProgressBar	Computed from engagement	Visual progress bar toward required hours
isEngagementComplete	boolean	Computed from totalHours >= REQUIRED	Controls whether completion badge is shown

Points Module

Variable	Type	DB Mapping	Purpose
spnPointAmount	JSpinner	points.amount	Number of points to award or deduct
cboTransactionType	JComboBox<String>	points.type ENUM	Award or Deduct selection
txtReason	JTextField	points.reason	Justification for the points transaction
spnTransactionDate	JSpinner	points.transaction_date	Date of the transaction
currentBalance	int	SUM(amount) with sign	Net points balance for the selected student
lblBalance	JLabel	Computed field	Large display label showing current balance
tblTransactions	JTable	points JOIN students	Color-coded transaction history list
tblLeaderboard	JTable	GROUP BY student_id ORDER BY balance DESC	Top 10 students ranked by net balance

Reports Module

Variable	Type	DB Mapping	Purpose
cboReportType	JComboBox<String>	N/A (selection)	Selects the type of report to generate
cboStudentFilter	JComboBox<String>	students.student_id	Selects a specific student or 'All Students'
txtaReportPreview	JTextArea	Assembled report text	Displays the generated report before export
dateFrom	LocalDate	Date filter parameter	Start date for date-range filtered reports
dateTo	LocalDate	Date filter parameter	End date for date-range filtered reports
selectedProgram	String	students.program	Filters report by CWTS, ROTC, or All
exportFilePath	String	java.io.File path	Destination file path chosen by JFileChooser

System-Wide Variables

util/Constants.java — System-Wide Constants

```
package util;

/**
 * Constants — Centralized system constants for NSTP Information System.
 * Import this class wherever you need system-wide configuration values.
 */
public class Constants {

    // Application Identity
    public static final String APP_TITLE      = "NSTP Information System";
    public static final String INSTITUTION    = "Pamantasan ng Lungsod ng Maynila";
    public static final String VERSION        = "1.0.0";

    // Database
    public static final String DB_NAME        = "nstp_infosystem";
    public static final int    DB_PORT        = 3306;

    // Business Rules
    public static final double ATTENDANCE_THRESHOLD = 75.0; // Below = at-risk
    public static final double REQUIRED_CWTS_HOURS = 20.0; // For completion
    public static final int    MAX_LOGIN_ATTEMPTS = 3;
    public static final int    DEMERIT_WARNING_THRESHOLD = 5; // Weighted total
    public static final int    NOTIF_CHECK_INTERVAL_MS = 60000; // 1 minute
    public static final int    UPCOMING_EVENT_DAYS = 3; // Notification trigger

    // UI Sizing
    public static final int    BTN_WIDTH      = 120;
```

```
public static final int    BTN_HEIGHT      = 35;
public static final int    TABLE_ROW_HEIGHT = 28;
public static final int    MAIN_FRAME_W    = 1200;
public static final int    MAIN_FRAME_H    = 700;

// Role Strings (match database ENUM exactly)
public static final String ROLE_ADMIN      = "Admin";
public static final String ROLE_CWTS_STUDENT = "CWTS_Student";
public static final String ROLE_ROT_C_STUDENT = "ROTC_Student";
public static final String ROLE_ROT_C_SENIOR = "ROTC_Senior";

private Constants() {} // Prevent instantiation
}
```


SECTION 7: IMPORTANT ENGINEERING STANDARDS

These standards are non-negotiable for the NSTP project. Every team member must follow them regardless of which module they are developing.

7.1 PreparedStatement-Only Policy

SQL INJECTION PREVENTION — MANDATORY

Every SQL query in the system **MUST** use PreparedStatement with ? placeholders. Never concatenate user input directly into a SQL string. Even one instance of string concatenation in a SQL query creates a security vulnerability that could expose the entire database.

Correct vs Incorrect SQL Handling

```
// =====
// WRONG — NEVER DO THIS — SQL INJECTION VULNERABILITY
// =====
String sql = "SELECT * FROM students WHERE full_name = '" + txtName.getText() + "'";
Statement stmt = conn.createStatement();
ResultSet rs = stmt.executeQuery(sql); // DANGEROUS

// =====
// CORRECT — ALWAYS USE PreparedStatement
// =====
String sql = "SELECT * FROM students WHERE full_name = ?";
try (PreparedStatement pst = conn.prepareStatement(sql)) {
    pst.setString(1, txtName.getText().trim());
    ResultSet rs = pst.executeQuery();
    // process results safely
}
```

7.2 Layer Separation Rules

Layer	Allowed To Call	NOT Allowed To Call
UI Panel (JPanel subclass)	DAO classes, Service classes, SessionManager, UIStyle	DBConnection directly, SQL statements, business logic
DAO class	DBConnection, model classes	Swing components, SessionManager, Service classes
Service class	DAO classes, model classes, Constants	Swing components, DBConnection directly
Model class	Nothing (pure data)	Everything else
SessionManager	Nothing (pure state)	Everything else
DBConnection	java.sql (JDBC only)	Everything else

7.3 Testing Checklist (Before Marking a Module Complete)

15. The module loads without errors when navigated to from the sidebar.
16. All fields accept valid input and display it correctly.
17. Saving a new record inserts a row in the correct database table (verified in phpMyAdmin).
18. Editing an existing record updates the correct row without creating duplicates.
19. Invalid input (empty fields, out-of-range values) shows a clear error message and does NOT save.
20. The table/view refreshes correctly after every save, edit, or delete operation.
21. The module respects role restrictions — a ROTC student cannot access admin panels.
22. Logout works correctly from this module — session is cleared, login screen appears.
23. The module handles a database connection failure gracefully with a user-friendly message.
24. Computed values (rates, balances, totals) match manual calculations from the database data.

7.4 Backup Protocol

- Before any schema change (adding a column, modifying an ENUM), export a .sql backup in phpMyAdmin.
- Save backups with timestamps: nstp_backup_2026_05_10.sql
- Keep at least 3 rolling daily backups in the shared project folder.
- Never modify the schema without telling all team members in advance.

SECTION 8: TEAM EXECUTION PLAN

This section assigns suggested roles and responsibilities across the 6-member development team (Famero, Capulong, Tinoko, Vallite, De Leon, Baterbonia). Roles may be adjusted by the team based on individual strengths.

8.1 Suggested Member Roles

Role	Primary Responsibilities	Key Files Owned
UI Lead / Login & Auth Developer	LoginFrame, authentication flow, SessionManager integration, MainFrame and StudentFrame shells, sidebar navigation, CardLayout wiring.	LoginFrame.java, MainFrame.java, StudentFrame.java
Admin Module Developer	UserManagementPanel, StudentManagementPanel, AnnouncementAdminPanel, FeedbackAdminPanel, ReportsPanel, export functionality.	ui.admin/* panels, ReportService.java
Student Module Developer	StudentDashboardPanel, AttendanceStudentPanel, EngagementStudentPanel, PerformanceStudentPanel, PointsStudentPanel, NotificationPanel, ProfilePanel.	ui.student/* panels
Database / DAO Developer	DBConnection, all DAO classes, SQL query design, batch operations, database error handling.	dao/*.java, util/DBConnection.java
Attendance & Performance Developer	AttendanceAdminPanel, AttendanceService, PerformanceAdminPanel, PerformanceService, DemeritAdminPanel, CalendarPanel.	AttendanceDAO, PerformanceDAO, DemeritDAO
Testing Lead / Documentation	Prepares test data, runs module tests, compiles bug list, maintains this document and the project's README, assists with demo preparation.	All test data SQL scripts

8.2 Daily Team Workflow

25. Daily standup (10 minutes): Each member states what they completed yesterday, what they are doing today, and any blockers.
26. Work on assigned module without touching other members' files unless coordinating.
27. Test your changes against the local database before end of day.
28. Export database backup and save to shared folder with today's timestamp.
29. Update the team's shared bug tracker or notes document with any issues found.

8.3 Database Ownership Rules

- Only the Database/DAO Developer creates new tables or modifies existing schema.

- All schema changes require a team announcement before implementation.
- Each developer writes and tests their own DAO methods but the DAO Developer reviews them.
- The DBConnection.java and SessionManager.java files are owned by the DB/DAO Developer and the UI Lead respectively. No one else edits these without permission.

SECTION 9: FINAL NOTE AND CONTINUATION PLAN

Document Scope

This master guide covers: Introduction to Java Swing (Section 1), Coding Standards and Guidelines (Section 2), System Feature Catalogue (Section 3), Software Requirements (Section 4), Phase 1 — Foundation Setup with full code (Section 5), Phase 2 — Variable Declarations and Global Structure (Section 6), Engineering Standards (Section 7), and Team Execution Plan (Section 8).

This document represents Phase 1 and Phase 2 of the NSTP Information System master development guide. The following topics will be covered in the continuation guide documents:

What Will Be Covered in Later Guides

- Phase 3 Guide: Authentication flow completion — `UserDAO.validateLogin()` with SHA-256 hash, role routing, SessionManager integration, failed attempt locking.
- Phase 4 Guide: Admin Dashboard module — summary cards, live statistics from MySQL, chart placeholders.
- Phase 5 Guide: Attendance Module — complete admin logging panel, batch insert, duplicate session guard, student view, dispute submission.
- Phase 6 Guide: Engagement Module (CWTS) and Performance Module (ROTC) — full code for both.
- Phase 7 Guide: Points Module, Demerit Module, Leaderboard — all with MySQL integration.
- Phase 8 Guide: Communication Modules — Announcements, Calendar, Notification Center with SwingTimer.
- Phase 9 Guide: Reports and Export Module — report generation, JFileChooser, .txt export.
- Phase 10 Guide: System Integration — CardLayout wiring, RBAC enforcement, cross-module data consistency testing.
- Phase 11 Guide: UI Polish — UIStyle application, input validation, SHA-256 password hashing, database error handling.
- Phase 12 Guide: Final Testing, Demo Data Preparation, Startup Checklist, Dry Run.

Important Reminder to the Team

Do not skip any phase of this guide. Each phase builds directly on the foundation of the previous one. Teams that skip Phase 1 and Phase 2 and jump straight to building panels will encounter JDBC connection errors, session management confusion, and naming inconsistencies that are difficult to fix later. Build the foundation correctly — the modules will follow naturally.

Group Fifth Unit — Famero | Capulong | Tinoko | Vallite | De Leon | Baterbonia

Pamantasan ng Lungsod ng Maynila | College of ISITM | May 2026

Prof. Darwin C. Co | NSTP Information System Capstone Project